# Assignment 3 Guidelines

You will continue the OverTheWire bandit challenge up to level 21 (i.e., finish the challenge from “Level 11 -> Level 12” to “Level 20 -> Level 21”, finally logging into level 21). You will also complete 3 problems about the security of various frameworks.

Do this assignment on your own. The assignment is due by **April 21st, 2026, at 11:59 PM Eastern time**. Zip each of your files into a directory with the naming convention **Lastname_Firstname.zip**. You can also use **tar.gz** (e.g., Lastname_Firstname.tar.gz).

- Make sure that your zip file can be uncompressed and the submitted code is readable with a text editor. Corrupted files will count zero points.
- A complete assignment zip file should contain 2 folders (one for each part) with the naming convention **Part_x** where x is the part’s number (e.g., Part_1). The folders should contain each part’s output files: one for part 1, and one for part 2, following the instructions below.
- The rubric is listed at the end of this document

## Part 1: Bandit_3 (40 pts)

As with the previous assignment, the goal is continuing with the "capture-the-flag" style wargame OverTheWire bandit challenge to familiarize yourself with the network principles discussed in class.

To do this, you will solve the next **10 levels** (i.e., finish the challenge from “Level 11 -> Level 12” to “Level 20 -> Level 21” such that your final task is to log into level 21) on the overthewire.org Bandit challenge website.

To track your progress, download the template of the part 1 report file, and fill in the requested sections. **Take screenshots of the series of commands you ran to find the password for each level**. Also, **take screenshots of you entering the password to reach the next level**. Your final report should include descriptions of all steps taken for logging into all accounts up to and including bandit21 on the OverTheWire bandit server.

1. A screenshot of the steps taken to obtain the password for the next level up to and including level 21 (i.e., log into level 21, but you do not need to solve for the level 22 password).
2. A screenshot showing the act of logging into all accounts up to and including bandit21 on the OverTheWire bandit server.
3. A brief description of what you did (demonstrate that you understand what the commands you ran did rather than simply restating the list of commands).

For example, if you were solving for level 0->1, your report would look like the following (without the redactions on the solutions):

1. A screenshot of steps to obtain the password for the next level (yours should not be redacted and should include the actual password and all steps to obtain it) :

2. A screenshot of successfully logging into the next level (it is ok if the password is not visible here, but there should be at least a partial banner to demonstrate that the attempt was successful. Also, include the ssh command in the screenshot.):

3. In your own words, give a description of what you did to obtain the password (do not simply list the commands, demonstrate that you understood what those commands accomplished).

Note that Bandit is an open system, and the goal of this assignment is to practice and develop your own skills, so be honorable and do not read walkthroughs. **Failure to report screenshots or manipulated screenshots will result in points deduction.**

## Submission Instructions for Part 1

As before, please submit a report file with a detailed description and screenshots of the solutions for each level. This report should be included in the **Lastname_Firstname** zip file, **Part_1** folder. You can submit the report in pdf, docx, and .odt format. Points will be deducted for missing content.

## Part 2: Security problems (60 pt)

The goal of this part of the assignment is for you to analyze and note the strengths and weaknesses of security protocols by answering questions about various problems described. **Be sure to read the questions carefully**, answer every part of the question, and explain your reasoning to earn full credit.

### 1. PayMe! (20 pts)

Prof. Pedantic, the esteemed Ineptitude Professor of Computer Science and Quackery at Warthogs University, hopes to become the next billionaire and decides to create an online money payment service similar to PayPal. His service, PayMe!, allows users to transfer money to other users of the system. To ensure that no fraudulent activity takes place, the PayMe! service stores the public key of each user. (You should assume that the sharing of the public key is secure; that is, the server has each user's correct public key.)

If Alice ("A") wishes to give X dollars to Bob ("B"), she sends the following message to the PayMe! service ("S"):

$$A \rightarrow S : A, B, X, n, \{X|n\}_{A-}$$

where n is a nonce, A− is Alice's private key, and $\{M\}_{K-}$ denotes a digital signature over M computed using the private key K−.

**QUESTIONS:**

(a). What is a nonce, and why does Prof. Pedantic include one in his protocol? (max 1 paragraph). What types of attacks does it prevent? (Explain **at least 2 attack examples**, max 2 paragraphs).

(b). Explain how an **active** adversary can exploit a weakness in Prof. Pedantic's protocol to steal money from an honest user, Alice. (Max 3 paragraphs).

## 2. PompousPass™ (20 pts)

Prof. Pedantic believes he has come up with a simple way of performing authentication. His system, PompousPass™, uses RSA signatures. Let $Id_x$ and $Pw_x$ respectively be the username and password for user x, and $(x^+, x^-)$ be the public/private keypair associated with user x. Assume that the server S knows the user's username ($Id_x$), password ($Pw_x$), and his public key ($x^+$). To authenticate to the server, x sends:

$$x \rightarrow S : Id_x, r, E_{x^-}(Id_x|Pw_x|r)$$

where r is a nonce.

The server decrypts the message using x's public key, and authentication proceeds if and only if (1) the transmitted password matches the password stored in the server's database and (2) the nonce is fresh.

**QUESTIONS:**

(a). Is this system secure? Why or why not? (Explain with examples. Max 2 paragraphs).

(b). Assuming that the parties share no keys other than their public keys (note that each party has their own private key which is not shared), provide a revised protocol that better protects the client's password from use. (Explain the modified protocols' steps).

(c). In practice, public-key cryptography is often used to distribute session keys, which are then used with symmetric algorithms. Why is this approach preferred over using solely public-key operations? (Max 1 paragraph).

## 3. Kerberos-ish (20 pts)

Dissatisfied with Kerberos (he's difficult to please), Prof. Pedantic proposes a simpler protocol called ChompChomp™ that eliminates the use of the TGS. Let $ID_C$ and $IP_C$ be the respective ID and IP address of the client, $ID_V$ be the ID of the service that the client wishes to access, $K_C$ be a pre-shared key between the KDC and client, $K_V$ be a pre-shared key between the KDC and service V, $K_{C,V}$ be a temporary key generated by the KDC during the course of the protocol, $E_X(M)$ be the encryption of M using key X, and lifetime be the lifetime of a ticket. Finally, let $A \rightarrow B : M$ denote the transmission of message M from A to B.

Prof. Pedantic's protocol works as follows:

1. Client → KDC: $ID_C$ , $ID_V$
2. KDC → client: $E_{K_c}(K_{C,V}$ , lifetime, $ticket_V$), where $ticket_V = E_{K_V}(K_{C,V}$ , $ID_C$ , $IP_C$ , lifetime).
3. Client → service V: $ticket_V$ , $E_{K_{C,V}}(ID_C, IP_C, t)$, where $t$ is the current time.
4. Service V → client: $E_{K_{c,v}}(t + 1)$

**QUESTIONS:**

(a). Does ChompChomp™ achieve client authentication? How (if yes) or why not (if no)? (Max 2 paragraphs)

(b). Does ChompChomp™ achieve service server authentication? How (if yes) or why not (if no)? (Max 2 paragraphs)

(c). What key goal of Kerberos does ChompChomp™ **not** achieve? Why not? (Max 2 paragraphs)

## Submission Instructions for Part 2

Your answers to the questions should be included in the **Lastname_Firstname** zip file, **Part_2** folder.

You will prepare a report that includes your answers following the given template for part 2.

If you didn't solve one of the 3 problems, feel free to leave it blank, but you may be able to get some partial credit for reasonable explanations. You can submit the report in pdf, docx, and .odt format.

## Rubric:

### Part 1: Bandit_3 (40 pts)

Find the password for level 12 (2 pts)

Log into level 12 (2 pts)

Find the password for level 13 (2 pts)

Log into level 13 (2 pts)

Find the password for level 14 (2 pts)

Log into level 14 (2 pts)

Find the password for level 15 (2 pts)

Log into level 15 (2 pts)

Find the password for level 16 (2 pts)

Log into level 16 (2 pts)

Find the password for level 17 (2 pts)

Log into level 17 (2 pts)

Find the password for level 18 (2 pts)

Log into level 18 (2 pts)

Find the password for level 19 (2 pts)

Log into level 19 (2 pts)

Find the password for level 20 (2 pts)

Log into level 20 (2 pts)

Find the password for level 21 (2 pts)

Log into level 21 (2 pts)

Points will be deducted for missing or forged content.

**Part 2: Security Problems (60 pts)**

1. PayMe! (20 pts)
    a. Correctly explains what a nonce is (5)
       Correctly explains why Pedantic uses the nonce (5)
    b. Correctly explains **at least two** types of attack is being prevented (5)
       Correctly explains how an attacker could exploit a weakness in the scheme (5)
2. PompousPass (20 pts)
    a. Correctly indicates whether the system is secure (5)
       Correctly justifies why the system is or is not secure (5)
    b. Suggests a reasonable new protocol which protects and prevents use of the client's password (make sure to adhere to the restrictions on protocols given in the question). (5)
    c. Correctly explains why aspects of both symmetric and asymmetric cryptography are used in practice (5)

3. Kerberos-ish (20 pts)
    a. Correctly indicates whether the client is authenticated (5)

       Correctly explains their answer regarding client authentication (5)
    b. Correctly indicates whether service server is authenticated (5)

       Correctly explains their answer regarding service server

       authentication (5)